

# Web Development

In-Class Exercise Sheet

HDip in Computing  
September 2025 Cohort

■■■ Prepared by: **Dr. Kislay Raj**

■ Total Time: **60 Minutes**

■ Questions: **6**

## ■ Topics Covered in This Exercise Sheet

|                             |                                                                                                                                                                                    |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HTML Forms & Input Types    | Building forms using <code>&lt;input&gt;</code> , <code>&lt;select&gt;</code> , and <code>&lt;button&gt;</code> elements with correct attributes.                                  |
| CSS Selectors & Box Model   | Styling navbars and layouts using <code>display:inline</code> , <code>margin:auto</code> , <code>background-color</code> , <code>text-decoration</code> , and <code>color</code> . |
| JavaScript DOM Manipulation | Using <code>document.getElementById()</code> , <code>.value</code> , <code>.textContent</code> and <code>createElement()</code> to interact with web pages dynamically.            |
| JS Form Validation & Regex  | Reading and debugging JavaScript validation logic, understanding <code>return true/false</code> and regular expressions.                                                           |
| LocalStorage API            | Saving and reading data in the browser using <code>localStorage.setItem()</code> and <code>localStorage.getItem()</code> .                                                         |
| Full-Stack Mini App         | Combining HTML structure, DOM methods and validation to build a working To-Do list application from scratch.                                                                       |

## ■ Suggested Time Allocation

| Question | Topic               | Difficulty       | Time           |
|----------|---------------------|------------------|----------------|
| Q1       | HTML Forms          | Easy             | 5 mins         |
| Q2       | CSS Navbar Styling  | Easy–Medium      | 8 mins         |
| Q3       | JavaScript DOM      | Medium           | 8 mins         |
| Q4       | Debug JS Validation | Medium           | 10 mins        |
| Q5       | LocalStorage        | Medium–Hard      | 15 mins        |
| Q6       | Mini To-Do App      | Hard / Challenge | 14 mins        |
|          | <b>Total</b>        |                  | <b>60 mins</b> |

## Question 1

■ 5 mins

### Build a Registration Form

● Easy

Topic: HTML Forms & Input Elements

Using what you have learned about HTML forms, create a simple student registration form inside the starter HTML below. The form must include all the fields listed.

- Full Name — text input with a label
- Age — number input with a label
- City — text input with a label
- A Submit button
- All inputs must have a matching label using the for attribute

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<title>Registration Form</title>

</head>

<body>

<!-- Write your form here -->


</body>

</html>
```

#### ■ What this question covers:

- HTML form structure: , , ,
- Input types: type="text", type="number"
- The for attribute on labels linking to input id values
- HTML page boilerplate: DOCTYPE, head, body

■ Hint: Remember: each needs a for= attribute that matches the id= of its input.

## Question 2

■ 8 mins

### Style a Navigation Bar with CSS

● Easy-Medium

Topic: CSS Selectors, Display & Colours

The HTML navbar below is unstyled. Write CSS in a `<style>` tag inside the `<head>` to make it look like a proper navbar.

- Navbar background colour: #333 (dark grey)
- Links must display side by side — not stacked vertically
- Remove the underline from all links
- Link text colour must be white
- Bonus: add a hover effect that changes the link colour to #00B4D8

```
<div class="navbar">

<ul>

<li><a href="#">Home</a></li>

<li><a href="#">About</a></li>

<li><a href="#">Contact</a></li>

</ul>

</div>

/* Write your CSS below */

.navbar { }

.navbar ul { }

.navbar ul li { }

.navbar ul li a { }

.navbar ul li a:hover { }
```

#### ■ What this question covers:

- CSS selectors: class selector, descendant selectors
- display: inline — placing list items side by side
- text-decoration: none — removing link underlines
- background-color and color properties
- :hover pseudo-class for interactive effects

■ Hint: The key property to make list items go side by side is display: inline on the element.

### Question 3

■ 8 mins

#### DOM Manipulation — Show a Message

● Medium

Topic: JavaScript DOM: `getElementById`, `textContent`

Complete the `showMessage()` JavaScript function so that when the button is clicked, it reads the value from the input field and displays it inside the `<p id="output">` paragraph.

- Get the value from the input field using `getElementById`
- Check the input is not empty — if it is, alert the user
- If not empty, set the `textContent` of the output paragraph
- Display: Hello, [name]! Welcome to Web Development.

```

<!DOCTYPE html>

<html lang="en">

<body>

<input type="text" id="myInput" placeholder="Type your name">

<button onclick="showMessage()">Show</button>

<p id="output"></p>

<script>

function showMessage() {

// Step 1: Get value from input

var name = _____;

// Step 2: Check if empty

if ( _____ ) {

alert("Please enter your name!");

return;

}

// Step 3: Display in output paragraph

document.getElementById("output")._____ =

"Hello, " + name + "! Welcome to Web Development.";

}

</script>

</body>

</html>

```

■ **What this question covers:**

- document.getElementById() — selecting HTML elements
- .value — reading the current value of an input field
- .textContent — writing text into a DOM element
- Basic if/else validation logic in JavaScript
- onclick event handler triggering a JavaScript function

■ *Hint: Use document.getElementById("myInput").value to get what the user typed.*

## Question 4

■ 10 mins

### Debug the Validation Function

● Medium

Topic: JavaScript Form Validation & Regex

The validateForm() function below has exactly 3 bugs. Read the code carefully, find each bug, and write the corrected version. A comment above each bug explains what it should do.

- Bug 1 is in the empty-name check — the condition is backwards
- Bug 2 is in the regex pattern — it allows the wrong characters
- Bug 3 is in the final return statement — it returns the wrong value

```
function validateForm() {

  var name = document.getElementById("username").value;

  var email = document.getElementById("email").value;


  // Bug 1: Should alert when name IS empty, not when it has content

  if (name != "") {

    alert("Please enter your name");

    return false;

  }


  // Bug 2: Regex should allow LETTERS only, not numbers

  var namePattern = /^[0-9]+$/;

  if (!namePattern.test(name)) {

    alert("Name must contain only letters");

    return false;

  }


  // Bug 3: Should return true on success so the form submits

  alert("Form submitted successfully!");

  return false;

}
```

#### ■ What this question covers:

- Comparison operators: == vs != in validation conditions
- Regular expressions (Regex): [A-Za-z\s]+ for letters only vs [0-9]+ for digits
- return true vs return false — controlling form submission
- Reading and understanding existing JavaScript code

→ Debugging logic errors (not syntax errors)

■ *Hint: For the regex: `[A-Za-z\s]+` means letters and spaces. `[0-9]+` means digits only.*

## Question 5

■ 15 mins

### Save and Load Data with LocalStorage

● Medium-Hard

*Topic: LocalStorage API: setItem, getItem*

Complete the two missing lines of code so that the student's name is saved to localStorage when the button is clicked, and automatically displayed when the page loads.

- Line 1 inside saveName(): save the name to localStorage with key "studentName"
- Line 2 outside the function: read "studentName" back from localStorage
- Test it: type a name, click Save, then refresh the page — the name should still appear
- Bonus: add a Clear button that removes the saved name using localStorage.removeItem()



```

<!DOCTYPE html>

<html lang="en">

<body>

<input type="text" id="nameInput" placeholder="Enter your name">

<button onclick="saveName()">Save Name</button>

<p id="display"></p>

<script>

function saveName() {

var name = document.getElementById("nameInput").value;

// Line 1: Save name to localStorage

_____

alert("Name saved!");

}

// Line 2: Read from localStorage when the page loads

var saved = _____;

if (saved) {

document.getElementById("display").textContent =

"Welcome back, " + saved + "!";

}

</script>

</body>

</html>

```

■ **What this question covers:**

- `localStorage.setItem(key, value)` — saving data to the browser
- `localStorage.getItem(key)` — reading saved data back
- Data persistence: data survives a page refresh
- Conditional logic: only display if data exists (truthy check)
- Understanding LocalStorage limitations: browser-specific, string only

■ *Hint: Line 1: `localStorage.setItem("studentName", name)` | Line 2: `localStorage.getItem("studentName")`*



## Question 6

■ 14 mins

### Build a To-Do List App

● Hard

Topic: Full Mini App — DOM, Validation, createElement

Complete the `addTask()` function to build a working To-Do list. When the button is clicked, a new task should appear as a list item on the page. The starter HTML and structure are provided — you write the JavaScript inside `addTask()`.

- Get the value from the `taskInput` field
- Validate: if the input is empty, alert the user and stop
- Create a new element using `document.createElement()`
- Set its `textContent` to the task value the user typed
- Append the new to the
- Clear the input field after adding the task
- Bonus: add a delete button inside each that removes it when clicked

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<title>My To-Do List</title>

</head>

<body>

<h2>My To-Do List</h2>

<input type="text" id="taskInput" placeholder="Enter a task">

<button onclick="addTask()">Add Task</button>

<ul id="taskList"></ul>

<script>

function addTask() {

// Step 1: Get the value


// Step 2: Validate – not empty


// Step 3: Create a new <li> element


// Step 4: Set its text


// Step 5: Append to the list


// Step 6: Clear the input


}

</script>

</body>

</html>
```

■ **What this question covers:**

- `document.createElement()` — creating new HTML elements with JavaScript
- `element.textContent` — setting the text of a created element
- `element.appendChild()` — adding a new element into the DOM
- `document.getElementById().value` — reading input field values
- Input validation: checking for empty strings before proceeding
- `input.value = ""` — clearing a field after an action (same concept as `form.reset()`)
- Combining all skills: HTML structure + CSS awareness + JS logic together

■ *Hint: Steps in order: get value → validate → `createElement('li')` → set `textContent` → `appendChild` → clear input.*